

2주 차 보고서

(2026년 1 월 4일 ~ 2026년 1 월 10일)

작성자

박신우

이번 주 한 일

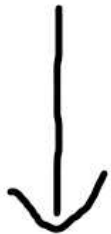
1.4 ~ 1.5 오브젝트 풀링 공부 ,시스템 구현

오늘 오브젝트 풀링에 대해서 공부를 하였고, 총알을 발사하거나 나중에 좀비를 소환하고 삭제할 때 SpawnActor / Destroy를 사용하면 성능이 저하될 수 있다고 생각해서 구현하였다. 이 시스템을 오늘 총알을 발사하는 것에만 적용하였다. 사실 지금 총알을 많이 쏘 봐야 성능이 저하되지 않아서 적용 전과 후의 차이를 크게 느끼진 못했다.

총알 발사



SpawnFromPool() -> 풀이 비어있다면 -> SpawnActor()로 총알 생성
풀에 총알이 있다면 -> 꺼내서 재사용



총알 날아감 -> (총알이 충돌함 or 일정시간이 지남) -> 풀에 보관

위와 같이 총알을 재사용하는 방식을 적용하였다.

이번 주 한 일

1.6 좀비 체력 적용

AFPSProjectile::OnHit()

총알이 좀비와 맞는 경우에 호출

```
if (OtherActor && OtherActor != this && OtherActor != MyOwner)
{
    // 데미지 적용 (20. f는 데미지 양, 조정 가능)
    UGameplayStatics::ApplyDamage(
        OtherActor, // 데미지 받을 액터
        20.f, // 데미지 양
        MyOwner ? MyOwner->GetInstigatorController() : nullptr, // 컨트롤러
        this, // 데미지 원인 액터(총알)
        nullptr // 데미지 타입
    );
    UE_LOG(LogTemp, Warning, TEXT("ammo damage to %s! "), *GetNameSafe(OtherActor));
}
```



ApplyDamage에 의해 좀비 액터에 데미지 전달

```
GetOwner()->OnTakeAnyDamage.AddDynamic(this, &UHealthComponent::DamageTaken);
```

UHealthComponent::BeginPlay()에서

이 컴포넌트를 장착하고 있는 액터가 데미지를 받았을 때

DamageTaken 함수를 호출하도록 설정

DamageTaken 함수에서 데미지만큼 체력을 빼게 설정하였다.

위의 방법으로 데미지를 주고 받을 수 있게 했다.

```
LogTemp: Warning: [HealthComponent] BP_BaseZombie_C_1 Health: 80.0
LogTemp: Warning: ammo damage to BP_BaseZombie_C_1!
LogSlate: Took 0.003892 seconds to synchronously load lazily loaded font '..../..
LogTemp: WeaponBase::Fire (Shooter: BP_FPSBaseCharacter_C_0, Weapon: BP_AR4_C_1
LogTemp: Warning: [HealthComponent] BP_BaseZombie_C_1 Health: 60.0
LogTemp: Warning: ammo damage to BP_BaseZombie_C_1!
LogTemp: WeaponBase::Fire (Shooter: BP_FPSBaseCharacter_C_0, Weapon: BP_AR4_C_1
LogTemp: Warning: [HealthComponent] BP_BaseZombie_C_1 Health: 40.0
LogTemp: Warning: ammo damage to BP_BaseZombie_C_1!
LogTemp: WeaponBase::Fire (Shooter: BP_FPSBaseCharacter_C_0, Weapon: BP_AR4_C_1
LogTemp: Warning: [HealthComponent] BP_BaseZombie_C_1 Health: 20.0
LogTemp: Warning: ammo damage to BP_BaseZombie_C_1!
```

이번 주 한 일

1.7 ~ 1.8 델리게이트 공부 , 적용

좀비가 총알과 충돌이 일어나 체력이 줄어들 때 다른 이펙트가 적용되도록 하려 한다. 좀비 체력을 감소하는 구조에서 체력 감소를 하며 아래 사진과 같은 구조로 작동하도록 하였다.

HealthComponent의 OnDamaged 델리게이트에 OnZombieDamaged 함수를 바인딩해서, 좀비가 데미지를 입으면 OnZombieDamaged가 자동 호출되도록 설정

```
void ABaseZombie::BeginPlay()
{
    Super::BeginPlay();

    if (HealthComponent)
    {
        HealthComponent->OnDamaged.AddDynamic(this, &ABaseZombie::OnZombieDamaged); // 데미지 입을 때 OnZombieDamaged를 호출
    }
}
```

UHealthComponent::DamageTaken 에서
체력을 감소시킨 후에 Broadcast 한다.

```
OnDamaged.Broadcast(Health, Damage);
```



ABaseZombie::OnZombieDamaged에서
액터의 위치에 이펙트 효과 발생



좀비와 총알이 충돌했을 때는 다른 이펙트가 발생하도록 해주었다.
(나중에 윤진이가 만든 피 튀는 이펙트 넣을 예정)

이번 주 한 일

1.9

현장프로젝트 수업 조별 과제와 발표 준비로 인해 못했다...

1.10 보고서 마무리, 코드 수정

위에서 델리게이트를 활용해서 좀비가 피격되었을 때는 다른 장애물과 충돌했을 때와 다른 이펙트가 발생하도록 하였는데, 보고서를 정리하면서 다시 코드를 보았는데 생각해 보니까, 좀비의 위치에 이펙트가 발생하는 것이 아니라 피격 위치에 이펙트를 발생하도록 해야 할 것 같다는 생각이 들었다.

또한, OnZombieDamaged 함수에서 피격 위치 정보를 제공하는 FHitResult 변수를 인자로 넘겨주어야 나중에 예를 들어 좀비 팔·다리가 분해되는 부위별 연출 기능을 추가할 때도 불필요하게 코드를 뜯어고치는 일이 없겠다는 생각이 들었다. 그래서 삽질했음을 깨닫고 코드를 다시 갈아엎고 있다.....

다음 주 할 일

차량 관련 기본적인 기능 구현.

좀비 기능 더욱 다듬기.

회의 진행하여 에셋 결정하기

특이사항

현장프로젝트교과를 듣다보니까 생각보다 시간이 별로 없었고, 힘들다는 핑계로 작업을 덜 진행하여서 지난 주에 목표했던 것보다 작업을 적게 진행하였다... 다음주에 더욱 힘을 내어 작업을 진행해야겠다.

2주 차 보고서

(2025년 1 월 4 일 ~ 2026년 1 월 10 일)

작성자

배주환

이번 주 회의 내용

이번 주 한 일

SpinLock, Sleep, Event, condition_variable, future, promise, packaged_task 등 다양한 Lock 구현 방식에 대해 공부하고 실습을 해보았다.

```
class SpinLock
{
public:
    void lock()
    {
        // CAS (Compare-And-Swap)

        bool expected = false;
        bool desired = true;

        while ( _locked.compare_exchange_strong(expected, desired) == false)
        {
            expected = false;

            //this_thread::sleep_for(std::chrono::milliseconds(100));
            this_thread::sleep_for(0ms);
            //this_thread::yield();
        }
    }

    void unlock()
    {
        _locked.store(false);
    }

private:
    atomic<bool> _locked = false;
};
```

```
void Producer()
{
    while (true)
    {
        {
            unique_lock<mutex> lock(m);
            q.push(100);
        }

        ::SetEvent(handle);

        this_thread::sleep_for(100ms);
    }
}

void Consumer()
{
    while (true)
    {
        ::WaitForSingleObject(handle, INFINITE);

        unique_lock<mutex> lock(m);
        if (q.empty() == false)
        {
            int32 data = q.front();
            q.pop();
            cout << data << endl;
        }
    }
}

int main()
{
    // 카널 오브젝트
    // Usage Count
    // Signal(피관볼) / Non-Signal(활관볼) << bool
    // Auto / Manual << bool
    handle = ::CreateEvent(NULL/*보안속성*/, FALSE/*bManualReset*/, FALSE/*bInitialState*/, NULL);

    thread t1(Producer);
    thread t2(Consumer);
}
```

```

while (true)
{
    // 1) Lock을 잡고
    // 2) 공유 변수 값을 수정
    // 3) Lock을 풀고
    // 4) 조건 변수를 통해 다른 스레드에게 통지

    {
        unique_lock<mutex> lock(m);
        q.push(100);
    }

    cv.notify_one();    // wait중인 스레드가 있으면 딱 한개만 깨운다

    //this_thread::sleep_for(100ms);
}

Consumer()
while (true)
{
    unique_lock<mutex> lock(m);
    cv.wait(lock, []() { return q.empty() == false; });
    // 1) Lock을 잡고
    // 2) 조건 확인
    // - 만족0 => 빠져 나와서 이어서 코드 진행
    // - 만족X => Lock을 풀어주고 대기 상태

    // 그런데 notify_one을 했으면 항상 조건식을 만족하는거 아닌가?
    // Spurious Wakeup(가짜 기상?)
    // notify_one을 할때 lock을 잡고 있는 것이 아니기 때문

    // std::future
    {
        // 1) deferred -> lazy evaluation 지연해서 실행하세요
        // 2) async -> 별도의 스레드를 만들어서 실행하세요
        // 3) deferred | async -> 둘 중 알아서 골라주세요

        // 언젠가 미래에 결과물을 뱉어줄거야
        std::future<int64> future = std::async(std::launch::async, Calculate);

        // TODO

        int64 sum = future.get(); // 결과물이 이제서야 필요하다!
    }

    // std::promise
    {
        // 미래(std::future)에 결과물을 반환해줄거라 약속(std::promise)해줘
        std::promise<string> promise;
        std::future<string> future = promise.get_future();

        thread t(PromiseWorker, std::move(promise));

        string message = future.get();
        cout << message << endl;

        t.join();
    }

    // std::packaged_task
    {
        std::packaged_task<int64(void)> task(Calculate);
        std::future<int64> future = task.get_future();

        std::thread t(TaskWorker, std::move(task));

        int64 sum = future.get();
        cout << sum << endl;

        t.join();
    }
}

```

간단하게 서버 프레임워크를 만들어서 DummyClient, GameServer, ServerCore로 분류해놓았다.

다음 주 할 일

프레임워크를 더 수정해가면서 컴퓨터 구조와 관련된 이론인 캐시, 메모리 모델, cpu 파이프라인에 대해 공부할 예정이다.

특이사항